

APUNTE 3

Permisos Especiales, ACLs y Modelos de Control de Acceso

Apunte 1 vimos el modelo básico de permisos en UNIX: tres categorías de acceso (propietario, grupo, otros) con tres tipos de permisos (lectura, escritura, ejecución). Este modelo es sencillo y funciona bien para la mayoría de los casos cotidianos. Sin embargo, en sistemas reales aparecen situaciones que requieren mecanismos más sofisticados.

Este apunte cubre tres niveles de complejidad creciente: primero los permisos especiales del sistema UNIX (SUID, SGID, Sticky Bit), luego las Listas de Control de Acceso (ACLs) que permiten granularidad por usuario, y finalmente los grandes modelos teóricos de control de acceso (DAC, MAC, RBAC) con sus implementaciones reales y una comparación entre sistemas operativos.

1. La Matriz de Control de Accesos

Antes de entrar en los mecanismos específicos, conviene entender el marco teórico que los unifica. En seguridad de sistemas operativos, el problema central es siempre el mismo: definir qué sujetos pueden realizar qué acciones sobre qué objetos.

La herramienta conceptual más directa para modelar esto es la matriz de control de accesos. Es una tabla donde las filas representan sujetos (usuarios o procesos), las columnas representan objetos (archivos, directorios, dispositivos, etc.) y cada celda indica qué acciones están permitidas.

Sujeto \ Objeto	/etc/passwd	/home/alice/	/var/www/	/etc/shadow
root	r, w, x	r, w, x	r, w, x	r, w
alice	r	r, w, x	—	—
bob	r	—	r, x	—
www-data	r	—	r, x	—

Este modelo es conceptualmente simple pero muy poderoso. Todo lo que no aparece explícitamente como permitido, está denegado — este es el principio de denegación por defecto (deny by default), que es una de las bases de la seguridad bien diseñada.

1.1 Implementaciones de la matriz

Una matriz completa de todos los usuarios versus todos los archivos del sistema sería enorme e inmanejable. En la práctica, la información se almacena de dos formas complementarias:

- **Por columnas — ACLs (Access Control Lists):** cada objeto almacena la lista de sujetos que pueden acceder a él y con qué permisos. Es el enfoque que usan los

sistemas de archivos modernos. Eficiente para responder: ¿quién puede acceder a este archivo?

- **Por filas — Listas de capacidades (Capability Lists):** cada sujeto almacena la lista de objetos a los que puede acceder. Eficiente para responder: ¿a qué puede acceder este usuario? Es el enfoque de las capabilities del kernel de Linux que vimos en el Apunte 2.

Los permisos básicos rwx de UNIX son una forma muy comprimida de ACL: en vez de listar cada usuario individualmente, agrupan en tres categorías (owner, group, others). Las ACLs POSIX que veremos más adelante rompen esa limitación.

1.2 El Monitor de Referencias

El componente del sistema operativo que aplica la matriz en tiempo real se llama Monitor de Referencias (Reference Monitor). Es el árbitro que intercepta cada intento de acceso a un recurso y decide si está permitido según la política vigente.

Para ser confiable, un monitor de referencias debe cumplir tres propiedades:

- **Completo:** debe mediar en todos los accesos, sin excepción. Ningún acceso puede saltar el control.
- **Verificable:** debe ser lo suficientemente simple como para poder auditarse y demostrarse que funciona correctamente.
- **Inviolable:** ningún proceso de usuario debe poder modificar o evadir el monitor.

En Linux, el monitor de referencias está implementado en el kernel a través del subsistema LSM (Linux Security Modules), que es la base sobre la que se construyen SELinux y AppArmor.

2. Permisos Especiales en UNIX/Linux

Los permisos especiales SUID, SGID y Sticky Bit son extensiones del modelo básico rwx. Modifican el comportamiento estándar de ejecución y creación de archivos, permitiendo casos de uso que los permisos simples no pueden resolver. Sin embargo, cada uno representa un riesgo si se usa incorrectamente.

2.1 SUID — Set User ID

Normalmente, cuando un usuario ejecuta un programa, el proceso resultante corre con los permisos de ese usuario. El bit SUID cambia esta regla: hace que el proceso corra con los permisos del propietario del archivo ejecutable, independientemente de quién lo ejecute.

El caso de uso clásico es el comando `passwd`. Para cambiar su contraseña, un usuario necesita modificar `/etc/shadow`, que solo root puede escribir. La solución: el binario `/usr/bin/passwd` pertenece a root y tiene el bit SUID activo. Cuando Alice ejecuta `passwd`, el proceso corre con UID efectivo 0 (root), aunque Alice tenga UID 1001.

```
# Ver que passwd tiene SUID activo
ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 59976 Nov 24 2022 /usr/bin/passwd
# ^
# La 's' en lugar de 'x' indica SUID activo

# Activar SUID en un archivo
```

```
chmod u+s archivo
# o en notación octal (el 4 al principio indica SUID)
chmod 4755 archivo    # resultado: rwsr-xr-x

# Si el archivo NO tiene permiso de ejecución, aparece 'S' (mayúscula)
chmod 4644 archivo    # resultado: rwSr--r-- ← SUID inactivo (sin x no tiene efecto)
```

UID real vs UID efectivo

Cuando SUID está activo, el proceso tiene dos identidades simultáneas:

- **UID real (RUID):** el usuario que ejecutó el programa (ej: alice, UID 1001). Indica quién eres realmente.
- **UID efectivo (EUID):** el usuario cuyos permisos se usan para las decisiones de acceso (ej: root, UID 0). Indica con qué privilegios operas.

```
// Programa de ejemplo para ver ambos UIDs
#include <stdio.h>
#include <unistd.h>
int main() {
    printf("UID real      (quién soy): %d\n", getuid());
    printf("UID efectivo (con qué privilegios): %d\n", geteuid());
    return 0;
}

// Compilar y activar SUID (como root):
// gcc -o setuid_demo setuid_demo.c
// chmod u+s setuid_demo

// Ejecutar como alice (UID 1001):
// UID real      (quién soy): 1001
// UID efectivo (con qué privilegios): 0 ← corre como root
```

SUID en scripts de shell (bash, python, etc.) está deshabilitado en Linux moderno por razones de seguridad. Solo funciona en binarios compilados. Si necesitas SUID en un script, la solución correcta es envolverlo en un binario C que lo llame.

Riesgos de SUID mal configurado

Si un archivo con SUID y propietario root tiene permisos de escritura para otros, cualquier usuario puede modificar su contenido y hacer que root ejecute código arbitrario la próxima vez que alguien lo llame. Esto es una escalación de privilegios directa.

```
# Configuración PELIGROSA — nunca hacer esto
-rwsrwxrwx 1 root root /usr/local/bin/mi_script
#      ^^^
#      Escritura para todos: cualquiera puede modificar el script
#      que luego se ejecutará con privilegios de root
```

```
# Configuración CORRECTA para un binario con SUID
-rwsr-xr-x 1 root root /usr/local/bin/mi_script
#   ^^^
# Solo root puede modificarlo
```

2.2 SGID — Set Group ID

El bit SGID se aplica a archivos ejecutables y directorios. SGID funciona de forma análoga a SUID, pero a nivel de grupo. Su comportamiento varía según se aplique a un archivo ejecutable o a un directorio.

SGID en archivos ejecutables

En archivos ejecutables, el bit SGID permite que el archivo se ejecute con los privilegios del grupo propietario del archivo, no con el grupo del usuario que ejecuta el programa. Es menos común que SUID pero sigue el mismo principio.

```
# Activar SGID en un archivo
chmod g+s archivo
# o en notación octal (el 2 al principio indica SGID)
chmod 2755 archivo # resultado: rwxr-sr-x
#   ^
# La 's' en la posición del grupo indica SGID
```

SGID en directorios — el caso más útil

Cuando SGID se aplica a un directorio, todos los archivos creados dentro heredan el grupo del directorio, en vez del grupo principal del usuario que los crea. Esto es extremadamente útil para directorios de trabajo compartido entre un equipo.

Ejemplo: El equipo de backend comparte el directorio /proyectos/backend. Sin SGID, cada desarrollador crea archivos con su grupo personal, y los demás no pueden acceder. Con SGID, todos los archivos creados en ese directorio pertenecen automáticamente al grupo backend.

```
# Configurar un directorio compartido con SGID
sudo mkdir /proyectos/backend
sudo chown root:backend /proyectos/backend
sudo chmod 2770 /proyectos/backend # rwxrws---
#   ^
# 2 = SGID activo

# Verificar
ls -ld /proyectos/backend
drwxrws--- 2 root backend 4096 Jun 10 14:30 /proyectos/backend
#   ^
# 's' en posición del grupo = SGID activo

# Ahora cualquier archivo creado dentro pertenecerá al grupo backend
```

```
touch /proyectos/backend/nuevo_archivo.py
ls -l /proyectos/backend/nuevo_archivo.py
-rw-rw-r-- 1 alice backend 0 Jun 10 14:31 nuevo_archivo.py
#           ^^^^^^^
#           El grupo es backend, no el grupo personal de alice
```

2.3 Sticky Bit

El Sticky Bit resuelve un problema específico de los directorios compartidos con escritura pública: si cualquiera puede escribir en el directorio, cualquiera puede también eliminar o renombrar los archivos de otros usuarios, aunque no sean suyos.

Con el Sticky Bit activo, solo el propietario del archivo, el propietario del directorio, o root pueden eliminar o renombrar archivos dentro del directorio. El resto puede crear sus propios archivos y leer los de otros (si los permisos lo permiten), pero no tocar los ajenos.

```
# El ejemplo clásico: /tmp
ls -ld /tmp
drwxrwxrwt 10 root root 4096 Jun 15 11:00 /tmp
#           ^
#           't' = Sticky Bit activo (con permiso de ejecución)
#           'T' = Sticky Bit activo (sin permiso de ejecución)

# Activar el Sticky Bit en un directorio
chmod +t /directorio_compartido
# o en notación octal (el 1 al principio indica Sticky Bit)
chmod 1777 /directorio_compartido # rwxrwxrwt
```

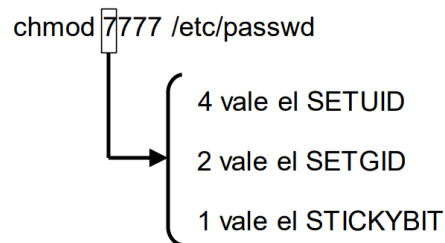
Ejemplo: /tmp es el ejemplo canónico. Todos los usuarios del sistema pueden crear archivos temporales en /tmp, pero nadie puede borrar los archivos de otro usuario. Sin Sticky Bit, Alice podría hacer `rm /tmp/archivo_de_bob` aunque ese archivo no le pertenezca.

2.4 Representación numérica de los permisos especiales

Los permisos especiales se representan como un cuarto dígito octal que va antes de los tres dígitos habituales:

Permiso	Valor octal	Ejemplo completo	Resultado
SUID	4	<code>chmod 4755</code>	<code>rwsr-xr-x</code>
SGID	2	<code>chmod 2755</code>	<code>rwxr-sr-x</code>
Sticky Bit	1	<code>chmod 1777</code>	<code>rwxrwxrwt</code>
SUID + SGID	6	<code>chmod 6755</code>	<code>rwsr-sr-x</code>

Estas representaciones se combinan con los permisos normales, que se muestran como tres dígitos numéricos (para propietario, grupo y otros), y se colocan antes de ellos en la notación octal.



Ejemplos:

- SUID activado: `chmod 4755` (permiso `rwxr-xr-x` con SUID).
- SGID activado: `chmod 2755` (permiso `rwxr-xr-x` con SGID).
- Sticky Bit activado: `chmod 1777` (típico para directorios como `/tmp`)

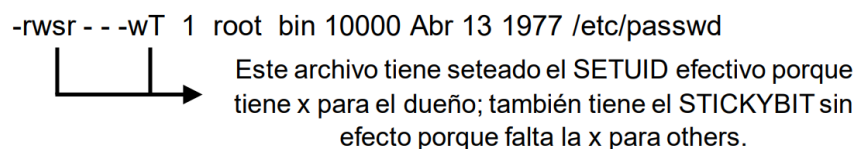
Comprobación de Estado de los Permisos Especiales

Para identificar si los permisos especiales están activos o desactivados, los administradores pueden utilizar varios métodos y comandos.

Visualización con ls -l:

- **SUID activado:** `rwsr-xr-x`
- **SGID activado:** `rwxr-sr-x`
- **Sticky Bit activado:** `rwxrwxrwt`

Si los permisos especiales no están activos, aparecerá una "**S**" mayúscula o "**T**" mayúscula, en lugar de la "**s**" o "**t**" minúsculas, indicando que estos bits están desactivados.



```
-rwsr--wT 1 root bin 10000 Abr 13 1977 /etc/passwd
```

Este archivo tiene seteado el SETUID efectivo porque tiene x para el dueño; también tiene el STICKYBIT sin efecto porque falta la x para others.

2.5 Auditoría de permisos especiales

Los permisos SUID y SGID son potenciales vectores de escalación de privilegios. Es importante saber cómo encontrarlos en el sistema y verificar que solo estén donde corresponde:

```
# Buscar todos los archivos con SUID activo en el sistema
find / -type f -perm -4000 2>/dev/null

# Buscar todos los archivos con SGID activo
find / -type f -perm -2000 2>/dev/null
```

```
# Buscar ambos al mismo tiempo
find / -type f \( -perm -4000 -o -perm -2000 \) 2>/dev/null

# Salida típica esperada (binarios legítimos del sistema):
/usr/bin/passwd      ← SUID, necesario para cambiar contraseñas
/usr/bin/sudo        ← SUID, necesario para elevar privilegios
/usr/bin/newgrp       ← SGID, necesario para cambiar de grupo
/usr/bin/wall         ← SGID, necesario para escribir a todos los terminales

# Cualquier binario SUID que NO reconozcas debe investigarse.
# Un binario SUID con propietario root fuera de /usr/bin o /bin
# es una señal de alerta importante.
```

Durante un pentest o una auditoría de seguridad, buscar binarios SUID es uno de los primeros pasos para intentar escalar privilegios. Herramientas como `linpeas.sh` lo hacen automáticamente. Mantener una lista de referencia de los SUID esperados en el sistema es una buena práctica.

3. ACLs — Listas de Control de Acceso

El modelo de permisos básico de UNIX tiene una limitación estructural importante: solo permite definir permisos para tres categorías fijas (propietario, grupo, otros). No hay forma nativa de decir "alice puede leer, bob puede leer y escribir, y carlos no puede acceder" sin cambiar el propietario o el grupo del archivo.

Las ACLs POSIX resuelven este problema permitiendo asignar permisos específicos a usuarios y grupos adicionales, independientemente de la propiedad del archivo.

3.1 Ver ACLs con `getfacl`

```
# Ver las ACLs de un archivo
getfacl informe.txt

# Salida sin ACLs adicionales (permisos básicos solamente):
# file: informe.txt
# owner: alice
# group: staff
# user::rw-      ← propietario (alice): lectura y escritura
# group::r--     ← grupo (staff): solo lectura
# other::---     ← otros: sin acceso

# Salida con ACLs adicionales configuradas:
# file: informe.txt
# owner: alice
# group: staff
# user::rw-
# user:bob:rw-   ← bob tiene acceso de lectura y escritura (ACL adicional)
# user:carlos:r-- ← carlos solo puede leer (ACL adicional)
# group::r--
```

```
# group:auditores:r-- ← el grupo auditores también puede leer
# mask::rw-          ← máscara efectiva (limita el máximo permiso de las ACLs)
# other::---
#
# El '+' al final en ls -l indica que hay ACLs activas
```

3.2 Configurar ACLs con setfacl

```
# Sintaxis básica:
# setfacl -m [tipo]:[nombre]:[permisos] archivo

# Dar permiso de solo lectura a bob
setfacl -m u:bob:r informe.txt

# Dar permiso de lectura y escritura a carlos
setfacl -m u:carlos:rw informe.txt

# Dar permiso de lectura al grupo auditores
setfacl -m g:auditores:r informe.txt

# Quitar todos los permisos ACL de un usuario específico
setfacl -x u:bob informe.txt

# Eliminar todas las ACLs adicionales del archivo
setfacl -b informe.txt

# Verificar que ls -l muestra el '+' indicando ACLs activas
ls -l informe.txt
-rw-rw-r--+ 1 alice staff 4096 Mar 31 14:00 informe.txt
#
# El '+' indica que hay ACLs adicionales
```

3.3 ACLs por defecto en directorios

Una característica muy útil es poder definir ACLs por defecto en directorios. Todos los archivos y subdirectorios creados dentro del directorio heredarán automáticamente esas ACLs.

```
# Configurar ACL por defecto: cualquier archivo nuevo en /proyectos/
# automáticamente dará acceso de lectura al grupo auditores
setfacl -d -m g:auditores:r /proyectos/
#
# -d indica que es una ACL por defecto (default)

# Verificar las ACLs por defecto del directorio
getfacl /proyectos/
# ...
# default:user::rwx
# default:group::r-x
# default:group:auditores:r-- ← heredada por nuevos archivos
# default:other::---
```

3.4 La máscara efectiva

Las ACLs tienen un elemento adicional llamado máscara (mask). La máscara actúa como un límite superior para los permisos de los usuarios y grupos adicionales (no afecta al propietario ni a "otros"). Es el permiso máximo que cualquier ACL de usuario o grupo puede tener.

```
# Ejemplo: mask limita los permisos efectivos
# ACL configurada: bob tiene rw
# mask: r--
# Permiso efectivo de bob: r-- (la mask limita a solo lectura)

# Cambiar la máscara manualmente
setfacl -m mask::r informe.txt

# La máscara se recalcula automáticamente cuando se modifica
# el permiso del grupo propietario con chmod
```

3.5 Quién puede configurar ACLs

- El propietario del archivo puede definir, modificar o eliminar ACLs sobre sus propios archivos.
- El usuario root puede modificar cualquier ACL del sistema.
- Usuarios con privilegios sudo pueden hacerlo si tienen los permisos necesarios.

3.6 Riesgos de seguridad con ACLs

La flexibilidad de las ACLs también introduce riesgos si no se administran con cuidado:

ACL sobre-privilegiada: si por error se da acceso rwx a un usuario sobre /etc/shadow, ese usuario puede leer todos los hashes de contraseñas del sistema. Siempre usar getfacl para auditar archivos sensibles.

```
# Ejemplo de configuración PELIGROSA (nunca hacer esto)
setfacl -m u:usuario_normal:rwx /etc/shadow
# Ahora usuario_normal puede leer, modificar y ejecutar /etc/shadow

# Auditoría: verificar las ACLs de archivos críticos
getfacl /etc/shadow
getfacl /etc/passwd
getfacl /etc/sudoers

# Buscar archivos con ACLs en un directorio
getfacl -R /etc/ 2>/dev/null | grep -v '^#' | grep -v '^$'
```

4. Modelos de Control de Acceso

Los mecanismos que vimos hasta ahora (permisos rwx, SUID, ACLs) son implementaciones concretas de modelos teóricos de control de acceso. Entender los modelos permite evaluar qué nivel de seguridad ofrece cada sistema y en qué contextos es apropiado usarlos.

Los tres modelos principales son DAC, MAC y RBAC. No son excluyentes: un sistema real puede usar varios simultáneamente.

4.1 DAC — Discretionary Access Control

En DAC (Control de Acceso Discrecional), el propietario de cada recurso decide quién puede acceder a él y con qué permisos. El sistema no impone una política centralizada: la decisión es discrecional, es decir, queda a criterio del propietario.

Es el modelo más común en sistemas de uso general. Todos los mecanismos que vimos hasta ahora (permisos UNIX, ACLs POSIX, permisos NTFS en Windows) son implementaciones de DAC.

Características

- **Flexible:** el propietario puede compartir o restringir acceso a voluntad.
- **Descentralizado:** no requiere un administrador central para gestionar cada permiso.
- **Propenso a errores:** la seguridad depende de que cada propietario configure bien sus permisos. Un error humano puede comprometer recursos.
- **Vulnerable a malware:** si un programa malicioso corre con los permisos de un usuario, puede acceder a todo lo que ese usuario puede acceder.

```
# DAC en UNIX/Linux: el modelo básico que ya conocemos
ls -l documento.txt
-rw-r--r-- 1 alice staff 1024 Jun 10 12:00 documento.txt
# Alice (propietaria) decidió: ella puede leer/escribir,
# el grupo staff puede leer, otros pueden leer.

# Alice puede cambiar esto cuando quiera:
chmod 600 documento.txt # ahora solo Alice puede acceder
chmod 664 documento.txt # ahora el grupo también puede escribir

# DAC con ACLs POSIX: más granular pero mismo modelo
setfacl -m u:bob:rw documento.txt
# Alice decidió darle acceso adicional a bob.
```

DAC en Windows — NTFS ACLs

Windows implementa DAC mediante las ACLs de NTFS, que son más ricas que los permisos UNIX básicos pero siguen el mismo modelo: el propietario controla el acceso. Las ACLs de NTFS permiten permisos como "lectura y ejecución", "modificar", "control total" y también tienen la noción de herencia (los objetos dentro de una carpeta heredan sus permisos).

Analogía entre UNIX y Windows: chmod → icacls, chown → takeown, getfacl/setfacl → Get-Acl/Set-Acl en PowerShell. El modelo conceptual es el mismo aunque la sintaxis difiere.

4.2 MAC — Mandatory Access Control

En MAC (Control de Acceso Obligatorio), las políticas de acceso las define el sistema o el administrador del sistema, no el propietario de cada recurso. Los usuarios no pueden cambiar los permisos de sus propios archivos si eso viola la política del sistema. Ni siquiera root puede evadir las restricciones MAC si el sistema está bien configurado.

MAC se usa en entornos donde la seguridad es crítica: sistemas militares, gubernamentales, servidores con datos altamente sensibles. El objetivo es proteger la información incluso contra errores o acciones deliberadas de usuarios con altos privilegios.

En MAC, cada sujeto y cada objeto tiene asignada una etiqueta de seguridad que indica su nivel de clasificación. Las políticas del sistema determinan qué operaciones están permitidas en función de esas etiquetas.

Modelo Bell-LaPadula — confidencialidad

Bell-LaPadula es el modelo MAC clásico para proteger la confidencialidad de la información. Define dos reglas fundamentales:

- **No read up (NRU):** un sujeto solo puede leer objetos de nivel igual o inferior al suyo. Un empleado no puede leer documentos clasificados como secreto si su nivel es solo confidencial.
- **No write down (NWD):** un sujeto solo puede escribir en objetos de nivel igual o superior al suyo. Un proceso con información secreta no puede escribirla en un archivo público (esto evita la filtración de información).

Nivel de clasificación	Puede leer	Puede escribir	Ejemplo
Top Secret (TS)	TS, S, C, U	Solo TS	Director de inteligencia
Secret (S)	S, C, U	Solo S o TS	Analista senior
Confidential (C)	C, U	Solo C, S, TS	Analista junior
Unclassified (U)	Solo U	Cualquier nivel	Usuario público

Modelo Biba — integridad

Biba es el modelo complementario a Bell-LaPadula, pero enfocado en proteger la integridad de los datos en vez de la confidencialidad. Sus reglas son las opuestas:

- **No read down:** un sujeto no puede leer objetos de nivel inferior (datos menos confiables podrían contaminar al sujeto).
- **No write up:** un sujeto no puede escribir en objetos de nivel superior (evita que datos no confiables modifiquen datos confiables).

Windows Integrity Control está basado en el modelo Biba. Define cuatro niveles: System, High, Medium y Low. Los procesos del sistema tienen nivel System o High, las aplicaciones del usuario tienen Medium, y los procesos potencialmente peligrosos (como el navegador procesando contenido web) tienen Low. Un proceso Low no puede modificar archivos de nivel Medium o superior.

Implementaciones de MAC en Linux

SELinux (Security-Enhanced Linux)

SELinux es la implementación de MAC más completa en Linux, desarrollada originalmente por la NSA. Funciona mediante etiquetas de seguridad asignadas a todos los objetos del sistema (archivos, procesos, puertos, etc.) y políticas que definen qué etiquetas pueden interactuar.

```
# Ver las etiquetas SELinux de archivos
ls -Z /var/www/html/
-rw-r--r--. root root unconfined_u:object_r:httpd_sys_content_t:s0 index.html
#
#                               ^^^^^^^^^^^^^^^^^^^^^^^^^^^
#                               La etiqueta 'httpd_sys_content_t' indica que
#                               solo procesos de Apache pueden acceder a este archivo

# Ver el estado actual de SELinux
getenforce
# Enforcing → activo, rechaza accesos no autorizados
# Permissive → activo, solo registra violaciones sin rechazarlas
# Disabled → desactivado

# Ver si un proceso tiene acceso denegado por SELinux
ausearch -m avc -ts recent | tail -20

# Ejemplo: aunque seas root, SELinux puede bloquear el acceso
# a archivos fuera de la política definida para ese proceso
```

AppArmor

AppArmor es la alternativa a SELinux usada por defecto en Ubuntu, Debian y sus derivados (incluyendo Kali Linux). En vez de etiquetas en los archivos, usa perfiles de seguridad por aplicación que definen exactamente a qué recursos puede acceder cada programa.

```
# Ver el estado de AppArmor
sudo apparmor_status

# Ver los perfiles activos
aa-status

# Ejemplo de perfil AppArmor para nginx (simplificado):
# /usr/sbin/nginx {
#   /var/www/html/** r,      ← puede leer archivos web
#   /var/log/nginx/** rw,    ← puede leer y escribir logs
#   /etc/nginx/** r,        ← puede leer su configuración
#   network tcp,            ← puede usar TCP
#   deny /etc/shadow r,     ← DENEGADO: no puede leer contraseñas
#   deny /home/** rw,       ← DENEGADO: no puede acceder a homes
# }

# Si nginx intenta acceder a /etc/shadow, AppArmor lo bloquea
# incluso si el proceso corre como root
```

Característica	SELinux	AppArmor
Mecanismo	Etiquetas en archivos y procesos	Perfiles por aplicación
Complejidad	Alta — curva de aprendizaje pronunciada	Media — más accesible
Por defecto en	RHEL, CentOS, Fedora, Android	Ubuntu, Debian, Kali, SUSE
Granularidad	Muy alta — control fino sobre cada recurso	Alta — por aplicación
Protege incluso a	root	root

4.3 RBAC — Role-Based Access Control

En RBAC (Control de Acceso Basado en Roles), los permisos no se asignan directamente a usuarios sino a roles. Los usuarios obtienen permisos al ser asignados a uno o más roles. Este modelo es el más usado en entornos empresariales con muchos usuarios y estructuras organizacionales claras.

La ventaja principal es la escalabilidad: en vez de gestionar los permisos de cada usuario individualmente, se gestionan los roles. Si se agrega un nuevo empleado, solo hay que asignarle el rol correspondiente y automáticamente tendrá todos los permisos necesarios.

Estructura de RBAC

- **Roles:** representan funciones organizacionales (administrador, desarrollador, auditor, usuario final, soporte técnico, etc.).
- **Permisos:** acciones sobre objetos, asignadas a los roles.
- **Usuarios:** asignados a uno o más roles. Heredan los permisos de todos sus roles.

Rol	Usuarios asignados	Permisos
admin	alice, root	Acceso total al sistema
dev	bob, carlos, diana	Leer/escribir en /proyectos, ejecutar tests
auditor	elena, auditores	Solo lectura en todos los logs y reportes
soporte	francisco	Leer logs, reiniciar servicios específicos
usuario_web	clientes externos	Acceso solo a la interfaz web pública

RBAC en Linux

Linux no tiene un sistema RBAC nativo en el espacio de usuario, pero se implementa a través de varias herramientas:

```
# RBAC básico mediante grupos en UNIX
# Crear el rol 'developers' como grupo
sudo groupadd developers
```

```
# Asignar usuarios al rol
sudo usermod -aG developers bob
sudo usermod -aG developers carlos

# Asignar permisos al rol (al grupo)
sudo chown :developers /proyectos/
sudo chmod 770 /proyectos/

# RBAC más sofisticado con sudo
# En /etc/sudoers se pueden definir 'roles' de comandos:
# %developers    ALL=(ALL) /usr/bin/systemctl restart nginx
# %auditores     ALL=(ALL) /usr/bin/journalctl, /bin/cat /var/log/*
# %soporte       ALL=(ALL) NOPASSWD: /usr/bin/tail -f /var/log/syslog
```

RBAC en bases de datos

MySQL y PostgreSQL tienen implementaciones nativas de RBAC que son más explícitas que el modelo de grupos de UNIX:

```
-- Crear un rol en MySQL
CREATE ROLE 'desarrollador';

-- Asignar permisos al rol
GRANT SELECT, INSERT, UPDATE ON app_db.* TO 'desarrollador';

-- Asignar el rol a usuarios
GRANT 'desarrollador' TO 'bob'@'localhost';
GRANT 'desarrollador' TO 'carlos'@'localhost';

-- Si se modifica el rol, todos los usuarios lo heredan automáticamente
GRANT DELETE ON app_db.logs TO 'desarrollador';
-- Ahora bob y carlos también pueden borrar logs, sin tocar sus cuentas
```

5. Puntos Críticos del Control de Acceso

Más allá de los mecanismos técnicos, hay cuestiones de diseño que todo sistema de control de acceso debe resolver. Estas preguntas aparecen tanto en exámenes como en la vida real.

5.1 Propagación de permisos

¿Puede un usuario que tiene acceso a un recurso otorgar ese acceso a otros? En DAC típicamente sí (es la base del modelo). En MAC no: los usuarios no pueden conceder permisos que el sistema no les permite conceder. Este es uno de los puntos donde DAC es más débil que MAC.

Ejemplo: Alice tiene permisos de lectura sobre un archivo confidencial. En un sistema DAC puro, Alice podría copiar el archivo a un directorio público y compartirlo con cualquiera. En un sistema MAC con Bell-LaPadula, el sistema bloquearía esa operación porque violaría la regla 'no write down'.

5.2 Revocación de permisos

Cuando se quita un permiso, ¿el efecto es inmediato o diferido? En sistemas simples como los permisos UNIX, la revocación es inmediata: el próximo intento de acceso se evalúa con los permisos actuales. Pero en sistemas con tickets o tokens de sesión (como Kerberos), un usuario puede seguir teniendo acceso hasta que su token expire, aunque se le haya revocado el permiso.

5.3 Permisos y modificación vs. contenido

Un punto sutil pero importante: si un usuario puede modificar un archivo ejecutable, puede insertar código malicioso que luego se ejecutará. Poder modificar el archivo no debería equivaler automáticamente a poder cambiar sus permisos. En UNIX estos son dos operaciones distintas: escribir en el archivo (permiso `w`) y cambiar sus permisos (solo el propietario puede hacerlo con `chmod`).

5.4 Objetos nuevos y permisos por defecto

Cuando se crea un objeto nuevo (un archivo, un proceso, una conexión de red), ¿qué permisos tiene? El sistema necesita una política por defecto. En UNIX eso es la `umask`. En MAC se suele heredar el nivel del proceso que lo crea. Si esta política por defecto es demasiado permisiva, introduce vulnerabilidades automáticamente.

Referencias Bibliográficas

- Silberschatz, A. — Operating System Concepts, Cap. 14 (Protection)
- OSTEP — Cap. 37 (Access Control)
- APUE (Advanced Programming in the UNIX Environment), Cap. 6
- Bishop, M. — Introduction to Computer Security (Bell-LaPadula, Biba, MAC/DAC)
- Baader, R. — Seguridad de la Información, DC-FCEyN-UBA (DAC, MAC, Matriz de accesos)
- Linux man pages: `setfacl(1)`, `getfacl(1)`, `chmod(1)`, `apparmor(7)`, `selinux(8)`